

Regex Cheatsheet v1.0.0 by [Serrin](#)

How to create a RegExp in Javascript	<code>new RegExp("A-Z", "g");</code> OR <code>/A-Z/g;</code>
Matches the character <code>a</code>	<code>/[a]/</code>
Matches the characters <code>a</code> , <code>b</code> , or <code>c</code>	<code>/[abc]/</code>
Matches all characters from uppercase <code>A</code> to uppercase <code>Z</code>	<code>/[A-Z]/</code>
Matches all characters from lowercase <code>a</code> to lowercase <code>z</code>	<code>/[a-z]/</code>
Matches all digits from <code>0</code> to <code>9</code>	<code>/[0-9]/</code>
Matches letters, numbers and hyphens	<code>/([A-Za-z0-9-]+)/</code>
<code>/d</code> flag: This specifies the start and the end of a match.	<code>"aaaabb".match(/(aa)(bb)/d) // ["aabb", "aa", "bb"]</code>
<code>/g</code> (global) flag: This ensures find all matches. Without the <code>/g</code> flag will only return the first match.	<code>"Is this all there is?".match(/is/); // ["is"]</code> <code>"Is this all there is?".match(/is/g); // ["is", "is"]</code>
<code>/i</code> (insensitive) flag: The makes a case-insensitive search.	<code>"Is this all there is?".match(/is/i); // ["Is"]</code> <code>"Is this all there is?".match(/is/gi); // ["Is", "is", "is"]</code>
<code>/m</code> (multiline) flag: This specifies a multiline match. Without <code>/m</code> set <code>^</code> match at the start of a string, <code>\$</code> match at the end of a string. With <code>/m</code> , <code>^</code> and <code>\$</code> will match the start and end of each line, in a multiline string.	<code>`Is this` all there is`.match(/^is/m); // ["is"]</code>
<code>/s</code> (dotAll) flag: This allows the <code>.</code> (dot) metacharacter to match any character, including line terminators (<code>\n</code> , <code>\r</code> , <code>\u2028</code> , <code>\u2029</code>). Without <code>/s</code> does not match line terminators.	<code>"Line\nLine.".match(/Line./g); // ["Line."]</code> <code>"Line\nLine.".match(/Line./gs); // ["Line\n", "Line."]</code>
<code>/u</code> (unicode) flag: This enables full Unicode support: 4-byte Unicode characters (like emojis or symbols) as two separate 2-byte code units.	<code>/\u{04DC0}.test("𐄀"); // false</code> <code>/\u{04DC0}/u.test("𐄀"); // true</code>
<code>/v</code> (unicodeSets) flag: This is an "upgrade" to the <code>/u</code> flag. The <code>\p</code> escape sequence matches strings, instead of just characters. Allow intersection, union, and subtraction syntaxes, and matching multiple Unicode characters. Using both <code>/u</code> and <code>/v</code> causes a <code>SyntaxError</code> .	<code>/\p{🍌}.test("Hello 🍌"); // false</code> <code>/\p{🍌}/v.test("Hello 🍌"); // true</code>
<code>/y</code> (Sticky) flag: This flag performs a "sticky" search that matches only from the <code>lastIndex</code> property. This flag ensures that subsequent matches start immediately after the previous one, without skipping characters.	<code>let pattern = /\w+/y;</code> <code>"abc def ghi".match(pattern); // ["abc"]</code> <code>pattern.lastIndex = 4; // Start match from position 4</code> <code>"abc def ghi".match(pattern); // ["def"]</code>
<code>a b</code> (OR) operator: The <code>/a b/</code> expression matches <code>a</code> OR <code>b</code> .	<code>"aa, bb, cc, dd".match(/aa bb dd/g); // ["aa", "bb", "dd"]</code> <code>"aa, bb, cc, dd".match(/aa ee/g); // ["aa"]</code> <code>"aa, bb, cc, dd".match(/ee ff/g); // null</code>
<code>^</code> (NOT) operator: Matches characters NOT in the range <code>A</code> to <code>Z</code> .	<code>"AbCdef".match(/[^A-Z]/g); // ["b", "d", "e", "f"]</code> <code>"AbCdef".match(/[A-Z]/g); // ["A", "C"]</code>
<code>.</code> (dot) metacharacter: Matches any character, except line terminators.	<code>"That's hot!".match(/h.t/); // "hat"</code>
<code>[\b]</code> Metacharacter: Matches backspace characters.	<code>"Lorem\bIpsum".search(/[\b]/); // 5</code>

Regex Cheatsheet v1.0.0 by [Serrin](#)

\d Metacharacter: Decimal digit characters. Same as [0-9]	<pre> /[\\d]/g.test("Abcd"); // false /[\\d]/g.test("Abcd42"); // true /[\\d]/g.test("42"); // true </pre>
\\D Metacharacter: Non-digit characters. Same as [^0-9]	<pre> /[\\D]/g.test("Abcd"); // true /[\\D]/g.test("Abcd42"); // true /[\\D]/g.test("42"); // false </pre>
\\s Metacharacter: Any whitespace characters. Same as [\\t\\n\\r\\f\\v]	<pre> "Arthur\\nDent".search(/\\s/); // 6 "ArthurDent".search(/\\s/); // -1 "\\r\\n".search(/\\s/); // 0 </pre>
\\S Metacharacter: Non-whitespace characters. Same as [^ \\t\\n\\r\\f\\v]	<pre> "Arthur\\nDent".search(/\\S/); // 0 "ArthurDent".search(/\\S/); // 0 "\\r\\n".search(/\\S/); // -1 </pre>
\\w Metacharacter: Alphanumeric characters. Same as [a-zA-Z0-9_]	<pre> "Arthur_Dent".search(/\\w/); // 0 "Arthur\\nDent".search(/\\w/); // 0 "!?,ArthurDent".search(/\\w/); // 3 "\\r\\n".search(/\\w/); // -1 </pre>
\\W Metacharacter: Non-alphanumeric characters. Same as [^a-zA-Z0-9_]	<pre> "Arthur_Dent".search(/\\W/); // -1 "Arthur\\nDent".search(/\\W/); // 6 "!?,ArthurDent".search(/\\W/); // 0 "\\r\\n".search(/\\W/); // 0 </pre>
\\0 Metacharacter: Matches NUL characters.	<pre> "Arthur\\0Dent".search(/[\\0]/); // 6 "Arthur Dent".search(/[\\0]/); // -1 </pre>
\\n Metacharacter: Matches new line characters.	<pre> "Arthur\\nDent".search(/[\\n]/); // 6 "Arthur Dent".search(/[\\n]/); // -1 </pre>
\\f Metacharacter: Matches form feed characters.	<pre> "Arthur\\fDent".search(/[\\f]/); // 6 "Arthur Dent".search(/[\\f]/); // -1 </pre>
\\r Metacharacter: Matches carriage return characters.	<pre> "Arthur\\rDent".search(/[\\r]/); // 6 "Arthur Dent".search(/[\\r]/); // -1 </pre>
\\t Metacharacter: Matches horizontal tabs (tabulators) characters.	<pre> "Arthur\\tIpsum".search(/[\\t]/); // 5 "Arthur Dent".search(/[\\t]/); // -1 </pre>
\\v Metacharacter: Matches vertical tab (tabulators) characters.	<pre> "Arthur\\vDent".search(/[\\v]/); // 6 "Arthur Dent".search(/[\\v]/); // -1 </pre>
\\b Metacharacter: Matches at the beginning or end of a word, not the full string. Search begin of a word: \\bxx and the end of a word: xx\\b	<pre> "Arthur Dent".search(/\\bDen/g); // 7 "Arthur Dent".search(/\\bden/g); // -1 "Arthur Dent".search(/ent\\b/g); // 8 "Arthur Dent".search(/enT\\b/g); // -1 </pre>
\\B Metacharacter: Matches NOT at the beginning/end of a word. Search NOT begin of a word: \\Bxx and NOT the end of a word: xx\\B	<pre> "Arthur Dent".search(/\\Bden/); // 8 "Arthur Dent".search(/\\BDen/); // -1 "Arthur Dent".search(/en\\B/); // 8 "Arthur Dent".search(/ent\\B/); // -1 </pre>

Regex Cheatsheet v1.0.0 by [Serrin](#)

<code>\ddd</code> Metacharacter: Match Latin character by an octal number.	<pre>"Arthur Dent".match(/\101/g); // ["A"] "Arthur Dent".match(/\102/g); // null</pre>																																									
<code>\xhh</code> Metacharacter: Match Latin characters by a hexadecimal number (hh).	<pre>"Arthur Dent".match(/\x41/g); // ["A"] "Arthur Dent".match(/\x42/g); // null</pre>																																									
<code>\uhhhh</code> Metacharacter: Match Unicode characters by a hexadecimal number (hhhh).	<pre>"Arthur Dent".match(/\u0041/g); // ["A"] "Arthur Dent".match(/\u0042/g); // null</pre>																																									
<code>\p{Unicode property}</code> Metacharacter: Matches characters with a Unicode character property.	<table><tr><th>Category</th><th>Abbr</th><th>Examples</th></tr><tr><td><code>\p{L}</code> Letter case-insensitive: A-Za-z from any languages (Áé)</td><td>L</td><td>ABcd</td></tr><tr><td><code>\p{Lu}</code> Uppercase_Letter</td><td>Lu</td><td>ABCD</td></tr><tr><td><code>\p{Ll}</code> Lowercase_Letter</td><td>Ll</td><td>abcd</td></tr><tr><td><code>\p{Lt}</code> Titlecase_letter</td><td>Lt</td><td>Ŋ (U+01CB)</td></tr><tr><td><code>\p{Lm}</code> Modifier_Letter</td><td>Lm</td><td>ˆ (U+02C6)</td></tr><tr><td><code>\p{Lo}</code> Other_Letter</td><td>Lo</td><td>not in Lu/Ll/Lt/Lm</td></tr><tr><td><code>\p{Nd}</code> Decimal_Number</td><td>Nd</td><td>1234</td></tr><tr><td><code>\p{Nl}</code> Letter_Number</td><td>Nl</td><td>I IV VII</td></tr><tr><td><code>\p{No}</code> Other_Number</td><td>No</td><td>¼ ½ ¾</td></tr><tr><td><code>\p{Sm}</code> Math_Symbol</td><td>Sm</td><td>+ = Σ</td></tr><tr><td><code>\p{Sc}</code> Currency_Symbol</td><td>Sc</td><td>\$ £ € ¥</td></tr><tr><td><code>\p{So}</code> Other_Symbol</td><td>So</td><td>© ®</td></tr></table>			Category	Abbr	Examples	<code>\p{L}</code> Letter case-insensitive: A-Za-z from any languages (Áé)	L	ABcd	<code>\p{Lu}</code> Uppercase_Letter	Lu	ABCD	<code>\p{Ll}</code> Lowercase_Letter	Ll	abcd	<code>\p{Lt}</code> Titlecase_letter	Lt	Ŋ (U+01CB)	<code>\p{Lm}</code> Modifier_Letter	Lm	ˆ (U+02C6)	<code>\p{Lo}</code> Other_Letter	Lo	not in Lu/Ll/Lt/Lm	<code>\p{Nd}</code> Decimal_Number	Nd	1234	<code>\p{Nl}</code> Letter_Number	Nl	I IV VII	<code>\p{No}</code> Other_Number	No	¼ ½ ¾	<code>\p{Sm}</code> Math_Symbol	Sm	+ = Σ	<code>\p{Sc}</code> Currency_Symbol	Sc	\$ £ € ¥	<code>\p{So}</code> Other_Symbol	So	© ®
Category	Abbr	Examples																																								
<code>\p{L}</code> Letter case-insensitive: A-Za-z from any languages (Áé)	L	ABcd																																								
<code>\p{Lu}</code> Uppercase_Letter	Lu	ABCD																																								
<code>\p{Ll}</code> Lowercase_Letter	Ll	abcd																																								
<code>\p{Lt}</code> Titlecase_letter	Lt	Ŋ (U+01CB)																																								
<code>\p{Lm}</code> Modifier_Letter	Lm	ˆ (U+02C6)																																								
<code>\p{Lo}</code> Other_Letter	Lo	not in Lu/Ll/Lt/Lm																																								
<code>\p{Nd}</code> Decimal_Number	Nd	1234																																								
<code>\p{Nl}</code> Letter_Number	Nl	I IV VII																																								
<code>\p{No}</code> Other_Number	No	¼ ½ ¾																																								
<code>\p{Sm}</code> Math_Symbol	Sm	+ = Σ																																								
<code>\p{Sc}</code> Currency_Symbol	Sc	\$ £ € ¥																																								
<code>\p{So}</code> Other_Symbol	So	© ®																																								
<code>String.prototype.includes(string, substring);</code>	<pre>/dent/g.test("Arthur Dent"); // false - case-sensitive /Dent/g.test("Arthur Dent"); // true - case-sensitive /dent/gi.test("Arthur Dent"); // true - case-insensitive</pre>																																									
<code>^</code> Metacharacter: Match from the beginning of a string, or the beginning of a line if the <code>m</code> flag is used.	<pre>/^arthur/g.test("Arthur Dent"); // false - case-sensitive /^Arthur/g.test("Arthur Dent"); // true - case-sensitive /^arthur/gi.test("Arthur Dent"); // true - case-insensitive</pre>																																									
<code>\$</code> Metacharacter: Match from the end of a string, or from the end of a line if the <code>m</code> flag is used.	<pre>/dent\$/g.test("Arthur Dent"); // false - case-sensitive /Dent\$/g.test("Arthur Dent"); // true - case-sensitive /dent\$/gi.test("Arthur Dent"); // true - case-insensitive</pre>																																									
<code>x(?:=y)</code> Lookahead matches <code>x</code> only if <code>x</code> is followed by <code>y</code> .	<pre>"Aa Bb".search(/Aa(?:= Bb)/g); //0</pre>	<pre>"Aa Bb".search(/Aa(?:= bb)/g); //-1</pre>																																								
<code>x(?:!y)</code> Negative Lookahead matches <code>x</code> if <code>x</code> is NOT followed by <code>y</code> .	<pre>"Aa Bb".search(/Aa(?:! Bb)/g); //-1</pre>	<pre>"Aa Bb".search(/Aa(?:! bb)/g); //0</pre>																																								
<code>(?<=y)x</code> Lookbehind matches <code>x</code> only if <code>x</code> is preceded by <code>y</code> .	<pre>"Aa Bb".search(/(?<=Aa)Bb/g); //3</pre>	<pre>"Aa Bb".search(/(?<=Aa)bb/g); //-1</pre>																																								
<code>(?<!y)x</code> Negative Lookbehind matches <code>x</code> only if <code>x</code> is NOT preceded by <code>y</code> .	<pre>"Aa Bb".search(/(?<!Aa)Bb/g); //-1</pre>	<pre>"Aa Bb".search(/(?<!aa)Bb/g); //3</pre>																																								
<code>(?flags:x)y</code> Group Modifiers	<pre>"Arthur Dent Ford".match(/(?i:arthur) dent/i); // ["Arthur Dent"]</pre>																																									

Regex Cheatsheet v1.0.0 by [Serrin](#)

(x) Capturing Groups - (abc) captures a group of characters - (abc)+ matches zero or more occurrence of the group - string.match() returns an array of results - match.groups() returns an array of matches	<pre>"abc, abc, abc".match(/(abc)+/); // ["abc", "abc"] "abc, abc, abc".match(/(xyz)+/); // null</pre>
?<name> Named Capturing Group - (?<firstName>\w+) captures a word and labels it firstName - (?<lastName>\w+) does the same for lastName - string.match() returns an array with a groups property - match.groups() returns an object: { firstName:"Arthur", lastName:"Dent" }	<pre>const match = "Name: Arthur Dent".match(/(?<firstName>\w+) (?<lastName>\w+)/); console.log(match.groups.firstName); // "Arthur" console.log(match.groups.lastName); // "Dent"</pre>

Quantifier	Same as		Min	Max	Details
+	x+	x{1,}	1	Infinity	The x+ matches at least one x.
*	*?	x{0,}	0	Infinity	The x* matches zero or more occurrences of x.
?	x?	x{0,1}	0	1	The x? matches zero or one occurrence of x.
{count}	x{count}		count	count	x{count} quantifier matches count occurence of x.
{min,}	x{min,}		min	Infinity	The x{min,} matches min or more occurrences of x. The min must be a number.
{min,max}	x{min,max}		min	max	The x{min,max} matches from min to max occurrence of x. The min and max must be a number.
"10, 100, 1000".match(/10+/g); // ["10","100","1000"] "10, 100, 1000".match(/100+/g); // ["100", "1000"] "10, 100, 1000".match(/1000+/g); // ["1000"] "10, 100, 1000".match(/10000+/g); // null "10, 100, 1000".match(/100000+/g); // null "10, 100, 1000".match(/32+/g); // null					"10, 100, 1000".match(/10{0}/g); // ["1", "1", "1"] "10, 100, 1000".match(/10{1}/g); // ["10", "10", "10"] "10, 100, 1000".match(/10{2}/g); // ["100", "1000"] "10, 100, 1000".match(/10{3}/g); // ["1000"] "10, 100, 1000".match(/10{4}/g); // null "10, 100, 1000".match(/32{0}/g); // null "10, 100, 1000".match(/32{1}/g); // null
"10, 100, 1000".match(/10*/g); // ["10","100","1000"] "10, 100, 1000".match(/100*/g); // ["10","100","1000"] "10, 100, 1000".match(/1000*/g); // ["100", "1000"] "10, 100, 1000".match(/10000*/g); // ["1000"] "10, 100, 1000".match(/100000*/g); // null "10, 100, 1000".match(/32*/g); // null					"10, 100, 1000".match(/10{0,}/g); // ["10", "100", "1000"] "10, 100, 1000".match(/10{1,}/g); // ["10", "100", "1000"] "10, 100, 1000".match(/10{2,}/g); // ["100", "1000"] "10, 100, 1000".match(/10{3,}/g); // ["1000"] "10, 100, 1000".match(/10{4,}/g); // null "10, 100, 1000".match(/32{0,}/g); // null "10, 100, 1000".match(/32{1,}/g); // null
"10, 100, 1000".match(/10?/g); // ["10", "10", "10"] "10, 100, 1000".match(/100?/g); // ["10", "100","100"] "10, 100, 1000".match(/1000?/g); // ["100", "1000"] "10, 100, 1000".match(/10000?/g); // ["1000"] "10, 100, 1000".match(/100000?/g); // null "10, 100, 1000".match(/32?/g); // null					"10, 100, 1000".match(/10{0,1}/g); // ["10", "10", "10"] "10, 100, 1000".match(/10{0,2}/g); // ["10", "100", "100"] "10, 100, 1000".match(/10{0,3}/g); // ["10", "100", "1000"] "10, 100, 1000".match(/10{0,4}/g); // ["10", "100", "1000"] "10, 100, 1000".match(/10{0,5}/g); // ["10", "100", "1000"] "10, 100, 1000".match(/32{0,1}/g); // null

Regex Cheatsheet v1.0.0 by [Serrin](#)

String: all uppercase	<code>/^[A-Z]+\$/.test(string)</code>
String: all uppercase, ignore non-letters: spaces, digits, or punctuation	<code>/^[A-Z]+\$/.test(str)</code> <code>/^(?=.*[A-Z])[^a-z]*\$/.test(str) // ignore non-letters: "123!!!"</code> <code>/^\p{Lu}+\$/u.test(string) // unicode safe: /u</code>
String: all lowercase	<code>/^[a-z]+\$/.test(str)</code> <code>/^(?=.*[a-z])[^A-Z]*\$/.test(str) // ignore non-letters: "123!!!"</code> <code>/^\p{Ll}+\$/u.test(string) // unicode safe: /u</code>
String all lowercase, ignore non-letters: spaces, digits, or punctuation	<code>/^[^A-Z]*\$/.test(string)</code>
Phone number: pattern match	<code>/^\d{3}-\d{2}-\d{4}\$/.test("123-45-6789")</code>
Phone number: Optional + for international format, starts with non-zero digit, total length: 8–15 digits (international standards like E.164)	<code>const PhoneNumber = (value) => /^\\+?[1-9]\\d{7,14}\$/.test(value);</code>
Phone number: Optional + for international format, total length: 8–15 digits	<code>const PhoneNumber = (value) => /^\\+?[0-9]\\d{7,14}\$/.test(value);</code>
Phone number: allows spaces, dashes, parentheses	<code>/^\\+?[1-9]\\d{7,14}\$/.test(string.replace(/[/\\s\\-()]/g, ""));</code>
Date (e.g. 21/3/2006)	<code>(\\d{1,2}\\\\/\\\\d{1,2}\\\\/\\\\d{4})</code>
Valid hexadecimal colour code	<code>(#?([A-Fa-f0-9]){3}(([A-Fa-f0-9]){3})?)</code>
Password: 8 to 15 chars, least 1 upper case, least 1 lower case, 1 digit.	<code>((?=.*\\d)(?=.*[a-z])(?=.*[A-Z]).{8,15})</code>
Email address: simple solution v1	<code>(\\w+@[a-zA-Z_]+?\\. [a-zA-Z]{2,6})</code>
Email address: simple solution v2 (better than v1)	<code>/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\. [a-zA-Z]{2,}\$/</code>
Email address: Unicode characters in the local part (before @) Unicode domains (may also appear in punycode, e.g. xn--...)	<code>const isValidEmailIntl = (x) => /^[\\p{L}\\p{N}._%\\-]+@([\\p{L}\\p{N}-]+\\.)+[\\p{L}]{2,}\$/u.test(x);</code>
Extra domain from url (handles ports, subdomains, different schemes: https:, http:, ftp:, etc.) Note: If possible, then please use these parsers: JavaScript : <code>new URL(url).hostname</code> Python : <code>urllib.parse.urlparse(url).hostname</code>	<code>"https://www.example.com/path?q=1".match(/^(?:[a-zA-Z][a-zA-Z0-9+.-]*:\\/\\/)?(^[\\/:\\n]+)/)[1]; // "www.example.com"</code> <code>"ftp://sub.domain.co.uk:21/path?q=1".match(/^(?:[a-zA-Z][a-zA-Z0-9+.-]*:\\/\\/)?(^[\\/:\\n]+)/)[1]; // "sub.domain.co.uk"</code>
Extract filename from an url	<code>const match = "https://example.com/path/file_name.tar.gz?download=true".match(/([\\\\/?#]+)(?:\\?.*)?\$/);</code> <code>if (match) { console.log(match[1]); } // "file_name.tar.gz"</code>
Extract filename from an url without extension	<code>const match = "https://example.com/images/photo.png".match(/([\\\\/?#]+)(?:\\. [^\\/?#.]+)(?:\\?.*)?\$/);</code> <code>if (match) { console.log(match[1]); } // "photo"</code>
HTML Tags	<code>(\\<(/?[^\\>]+)\\>)</code>
HTML Tags: Extract Specific Tag (Example: div)	<code><(/?div)\\b[^>]*></code>
HTML Tags: all div tags	<code>/<div\\b(?:[>'"] "[^"]*" '['']*')*>/gi</code>
HTML Tags: Extract Only Tag Names (Without Attributes)	<code><\\/?([a-zA-Z][a-zA-Z0-9-]*)\\b[^>]*></code>
HTML Tags: Extract Only Opening Tags (Not Closing)	<code><([a-zA-Z][a-zA-Z0-9-]*)\\b[^>]*?></code>
HTML Tags: Extract Closing Tags Only	<code><\\/?([a-zA-Z][a-zA-Z0-9-]*)></code>

Regex Cheatsheet v1.0.0 by [Serrin](#)

HTML tags: collect urls from the A tags and can handle malformed HTML properties

```
const html = "<a href='https://example.com'>Example</a><a class='x' href='https://google.com'>Google</a><a href=/relative/path>Relative</a>";
const regex =
  /<a\b(?:\bhref\s*=\s*(?:"([^\"]*)"|'([^']*)'|([^\s]+)))/gi;
const links = [];
let match;
while ((match = regex.exec(html)) !== null) {
  links.push(match[1] || match[2] || match[3]);
} /* ["https://example.com", "https://google.com",
"/relative/path"] */
```

```
const strIncludes = (str, substr, caseInsensitive = false)
=> new RegExp(String(substr), caseInsensitive ? "gi" :
"g").test(String(str));
```

```
const strStartsWith = (str, substr, caseInsensitive = false)
=> new RegExp(`^${String(substr)}`, caseInsensitive ? "gi" :
"g").test(String(str));
```

```
const strEndsWith = (str, substr, caseInsensitive = false)
=> new RegExp(`${String(substr)}$`, caseInsensitive ? "gi" :
"g").test(String(str));
```

```
/* test cases - all passed */
const { assert } = console;
assert(strIncludes("Arthur Dent", "Dent"), "1");
assert(strIncludes("Arthur Dent", "dent", true), "2");
assert(!strIncludes("Arthur Dent", "dent"), "3");
assert(strStartsWith("Arthur Dent", "Arthur"), "4");
assert(strStartsWith("Arthur Dent", "arthur", true), "5");
assert(!strStartsWith("Arthur Dent", "arthur"), "6");
assert(strEndsWith("Arthur Dent", "Dent"), "1");
assert(strEndsWith("Arthur Dent", "dent", true), "2");
assert(!strEndsWith("Arthur Dent", "dent"), "3");
```